ONLINE BANKING SYSTEM

TEAM MEMBERS:

- MADHAN KUMAR M
- SUBASH M
- MOHAMMED RIYAZ A
- PRAVEEN KUMAR P
- SANTHOSH KUMAR S

ONLINE BANKING SYSTEM

Aim:

To design and develop a database-driven Online Banking Management System that enables users to register, log in, manage their bank accounts, perform transactions, and track their financial activity using a MongoDB database and Node.js backend.

Database Design:

1. **Accounts Collection :**

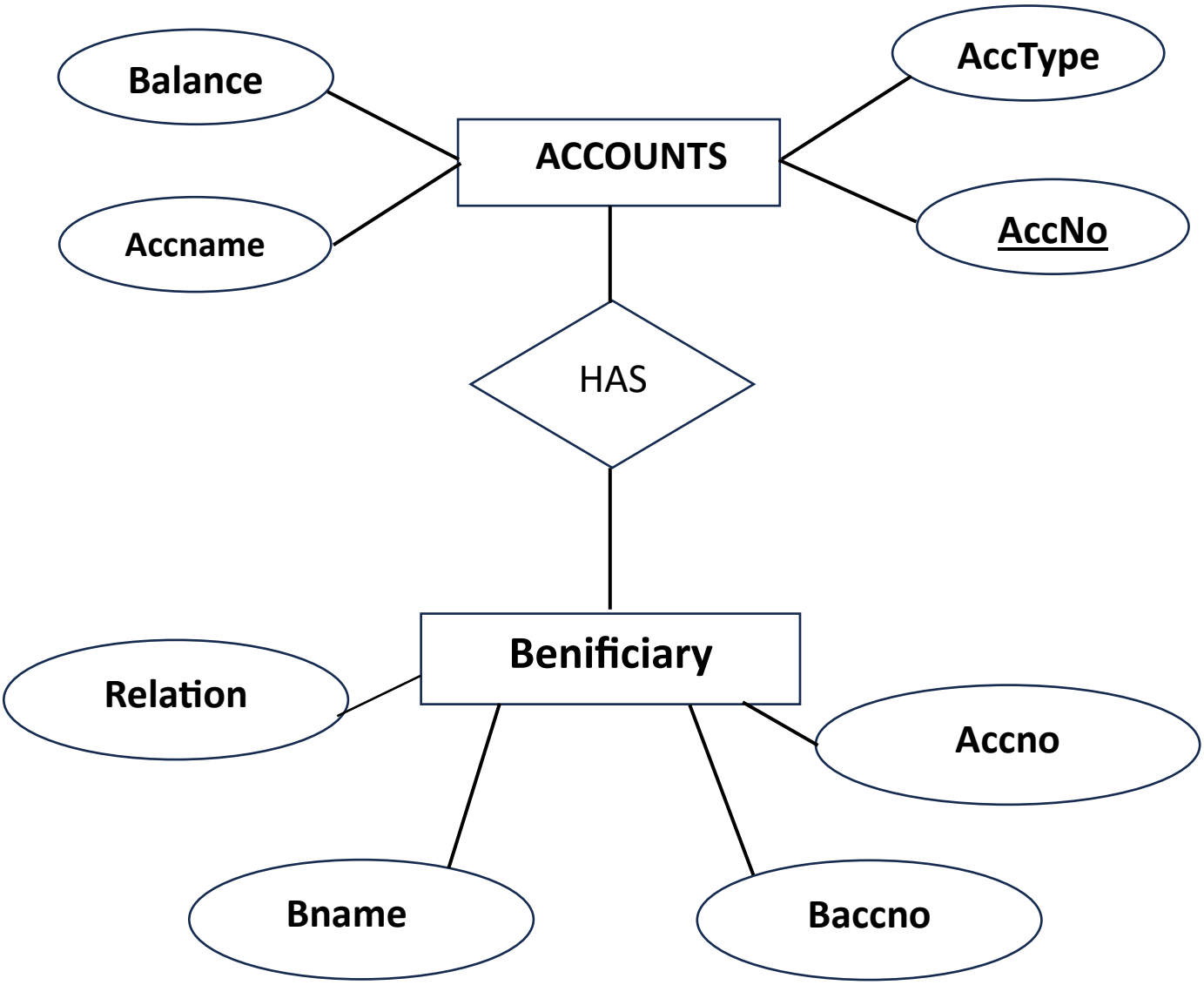
```
{
  accno: Number,
  accname: String,
  balance: Number,
  accounttype: String
}
```

2. **Userlogins Collection:**

```
{
  accno: Number,
  username: String,
  password: String
}
```

3. **Transactions Collection :**

```
{
  accno: Number,
  type: "credit" | "debit",
  amount: Number,
  description: String,
  date: Date
}
```


Balance
AccType
ACCOUNTS
Accname
AccNo
HAS
Benificiary
Relation
Accno
Bname
Baccno

Procedure :

1. Database Connection:

- Connected MongoDB using Mongoose with Express.

2. User Registration:

- Frontend form collects accno, username, password
- Data stored in userlogin and accounts

3. Login Authentication:

- Backend checks user credentials from userlogin

4. Dashboard Display:

- Retrieves data from accounts using accno

5. Deposit Functionality:

- User enters amount to deposit
- accounts.balance updated
- New transaction of type credit added to transactions

6. Withdraw Functionality:

- User enters amount to withdraw
- Balance checked before deducting
- accounts.balance updated and a debit transaction added

7. Money Transfer:

- Input: fromAcc, toAcc, and amount
- Amount debited from sender, credited to receiver
- Two entries in transactions: one debit and one credit

8. Transaction History:

- Queries all transactions with user's accno
- Displays on frontend in tabular format

9. Account Deletion:

- Deletes records from accounts, userlogin, and transactions

10. Validation Checks:

- No transfer to same account
- No withdrawal more than available balance
- Required field checks in both frontend and backend

11. Frontend Pages:

- Login, Register, Dashboard, Deposit, Withdraw, Transfer, History
- Styled with modern CSS gradients and card layouts

12. Error Handling:

- Try-catch blocks on backend
- Displays custom styled error/success messages

13. Navigation:

- Each action page has a button to go back to dashboard

14. Dynamic Routes:

- Used req.params to send user to their personal dashboard /dashboard/:accno

15. Transaction Timestamp:

- Used JavaScript Date object to record transaction date

Modules and Features

1. User Registration and Authentication

- *Account Creation: Users can create a new bank account by providing necessary details.*
- *Login: Users can log in using their credentials to access their dashboard.*
- *Admin Login: Administrators have separate login credentials to access administrative functionalities.*

2. Account Management

- *View Balance: Users can view their current account balance.*
- *Deposit Funds: Users can deposit money into their account.*
- *Withdraw Funds: Users can withdraw money, subject to sufficient balance.*
- *Fund Transfer: Users can transfer funds to another account within the system.*
- *Transaction History: Users can view a detailed history of their transactions.*

3. Administrative Controls

- *View All Accounts: Admins can view a list of all user accounts.*
- *Filter Accounts: Admins can filter accounts based on account type (e.g., Savings, Current).*
- *Delete Accounts: Admins can delete user accounts as necessary.*

Front End Code :

Opening page:

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8" />
<meta name="viewport" content="width=device-width, initial-scale=1.0"/>
<title>Bank Login</title>
<style>
body {
background: linear-gradient(135deg, #ffd6a5, #fdffb6); /* soft pastel yellow-orange */
font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
```

```
display: flex;
align-items: center;
justify-content: center;
height: 100vh;
margin: 0;
color: #555;
}

.input[type="text"]:focus,
input[type="password"]:focus {
border-color: #f7b733; /* brighter orange */
outline: none;
box-shadow: 0 0 10px 3px rgba(247, 183, 51, 0.5);
background-color: #fff8d6;
}

button {
width: 100%;
padding: 16px;
background: linear-gradient(135deg, #f7b733, #fceabb); /* golden
gradient */
border: none;
border-radius: 14px;
font-size: 18px;
font-weight: 700;
color: #7a4e00;
cursor: pointer;
transition: background 0.4s ease, transform 0.25s ease;
box-shadow: 0 8px 20px rgba(247, 183, 51, 0.35);
}
```

```
</style>
</head>
<body>
<div class="header">
<h1>STATE BANK OF INDIA</h1>
</div>
<div class="login-container">
<h2>Online Banking Login</h2>
<form id="loginForm" action="/login" method="post">
<input type="text" name="username" placeholder="Enter
username" required />
<input type="password" name="password"
placeholder="Password" required />
<button type="submit">Login</button>
</form>
<div class="signup-link">
Don’t have an account? <a href="/register">Sign up</a>
</div>
</div>
</body>

</html>
```

Loginpage :

```
<!DOCTYPE html>
<html>
<head>
<title>Login</title>
<style>
```

```
body {
font-family: Arial;
background: #fff9db;
display: flex;
justify-content: center;
align-items: center;
height: 100vh;
}
input, button {
width: 100%;
padding: 10px;
margin: 10px 0;
border-radius: 6px;
border: 1px solid #ccc;
}
button {
background: #f39c12;
color: white;
border: none;
}
button:hover {
background: #e67e22;
}
</style>
</head>
<body>
<div class="box">
<h2>Login</h2>
<form action="/login" method="POST">
```

```
<input type="text" name="username" placeholder="Username"
required>
<input type="password" name="password"
placeholder="Password" required>
<button type="submit">Login</button>
</form>
</div>
</body>
</html>
```

New register page :

```
<!DOCTYPE html>
<html>
<head>
<title>Register</title>
<style>
body {
font-family: Arial;
background: #fff9db;
display: flex;
justify-content: center;
align-items: center;
height: 100vh;
}
button {
background: #f39c12;
color: white;
border: none;
}
button:hover {
background: #e67e22;
```

```
}
</style>
</head>
<body>
<div class="box">
<h2>Register</h2>
<form action="/reg" method="POST">
<input type="number" name="accno" placeholder="Account Number" required>
<input type="text" name="username" placeholder="Username" required>
<input type="password" name="password" placeholder="Password" required>
<button type="submit">Create Account</button>
</form>
</div>
</body>
</html>
```

Admin Page :

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>WELCOME SBI</title>
  <link rel="stylesheet" href="basic.css">
```

```
</head>
<body>
   <a href="home.html" class="logout-btn">⏻ Log Out</a>
  <h1 class="main-heading">STATE BANK OF INDIA</h1>

   <div class="container">
      <h2>Welcome Admin</h2>
      <button><a href="/regform">New Account</a></button>
                  <button><a href="view.html">View Account
Details</a></button>
                        <button><a href="search.html">Search
Customers</a></button>
                                                <button><a
href="update/update.html">Transaction</a></button>
      <button><a href="delete.html">Delete Account</a></button>
   </div>
</body>

</html>
```

Backend Code:

```
// server.js
const express = require('express');
const mongoose = require('mongoose');
const path = require('path');
```

```
const app = express();
const port = 5000;

// Middleware
app.use(express.urlencoded({ extended: true }));
app.use(express.static(__dirname));

// Connect to MongoDB
mongoose.connect('mongodb://127.0.0.1:27017/banking', {
  useNewUrlParser: true,
  useUnifiedTopology: true
});

const db = mongoose.connection;
db.on('error', console.error.bind(console, 'Connection error:'));
db.once('open', () => {
  console.log('MongoDB connected');
});

// Schemas
const accountSchema = new mongoose.Schema({
  accno: Number,
  accname: String,
  balance: Number,
  accounttype: String
});

const loginSchema = new mongoose.Schema({
  accno: Number,
  username: String,
```

```
  password: String
});

const transactionSchema = new mongoose.Schema({
  accno: Number,
  type: String,
  amount: Number,
  description: String,
  date: { type: Date, default: Date.now }
});

// Models
const Account = mongoose.model('Account', accountSchema);
const UserLogin = mongoose.model('UserLogin', loginSchema);
const Transaction = mongoose.model('Transaction',
transactionSchema);

// Routes

// Home page
app.get('/', (req, res) => {
  res.sendFile(path.join(__dirname, 'home.html'));
});

// Register page
app.get('/register', (req, res) => {
  res.sendFile(path.join(__dirname, 'register.html'));
});

// Registration logic
```

```
app.post('/reg', async (req, res) => {
  const { accno, username, password } = req.body;

  const accountExists = await Account.findOne({ accno: accno });
  if (accountExists) return res.send("Account number already
exists");

  await Account.create({
    accno,
    accname: username,
    balance: 0,
    accounttype: "Savings"
  });

  await UserLogin.create({ accno, username, password });

  res.send("Registration successful!");
});
```

// Login logic

```
app.post('/login', async (req, res) => {
  const { username, password } = req.body;

  const user = await UserLogin.findOne({ username, password });
  if (!user) return res.send("Invalid login");

  res.redirect(/dashboard/${user.accno});
});
```

// Dashboard

```
app.get('/dashboard/:accno', async (req, res) => {
  const accno = req.params.accno;
  const account = await Account.findOne({ accno: accno });

  if (!account) return res.send("Account not found");

  res.send(`
    <h1>Welcome, ${account.accname}</h1>
    <p>Balance: ₹${account.balance}</p>
    <a href="/transfer.html">Transfer Money</a>
    <a href="/transactions/${accno}">View Transactions</a>
  `);
});
```

// Transfer money

```
app.post('/transfer', async (req, res) => {
  try {
    const { fromAcc, toAcc, amount } = req.body;
    const transferAmount = Number(amount);

    const from = await Account.findOne({ accno: fromAcc });
    const to = await Account.findOne({ accno: toAcc });

    if (!from || !to) return res.send("Account not found");
    if (from.balance < transferAmount) return res.send("Insufficient
balance");
 from.balance -= transferAmount;
    to.balance += transferAmount;
   await from.save();
    await to.save();
```

```
    await Transaction.create([
      {
        accno: fromAcc,
        type: "debit",
        amount: transferAmount,
        description: Transfer to ${toAcc}
      },
      {
        accno: toAcc,
        type: "credit",
        amount: transferAmount,
        description: Received from ${fromAcc}
      }
    ]);
  res.send("Transfer successful");
  } catch (err) {
    console.error(err);
    res.status(500).send("Server error");
  }
});
```

// Transaction History

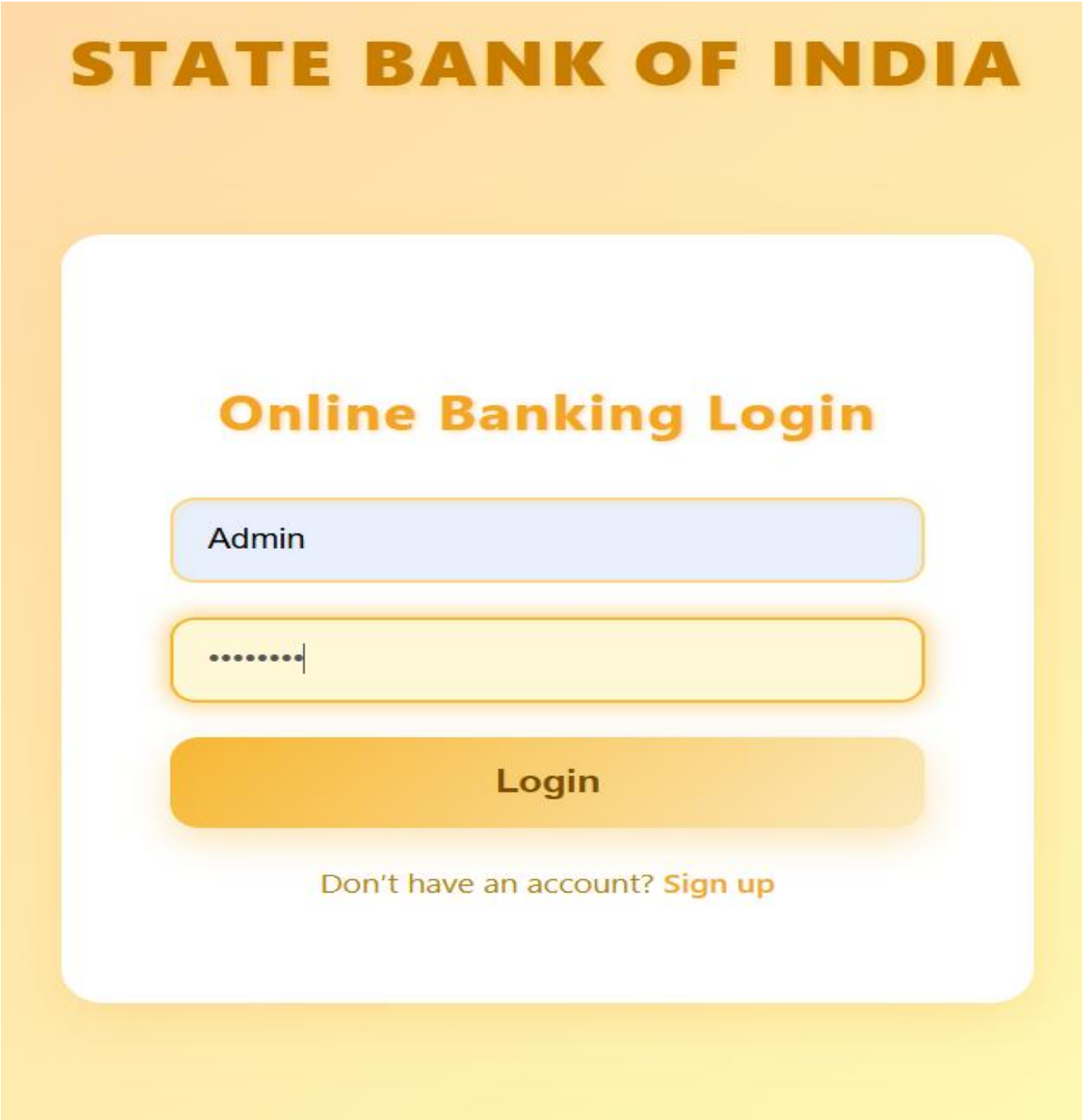
```
app.get('/transactions/:accno', async (req, res) => {
  const accno = req.params.accno;
  const transactions = await Transaction.find({ accno });

  let output = "<h2>Transaction History</h2><ul>";
  transactions.forEach(tx => {
    output += <li>${tx.date.toDateString()} - ${tx.type} -
₹${tx.amount} - ${tx.description}</li>;
  });
```

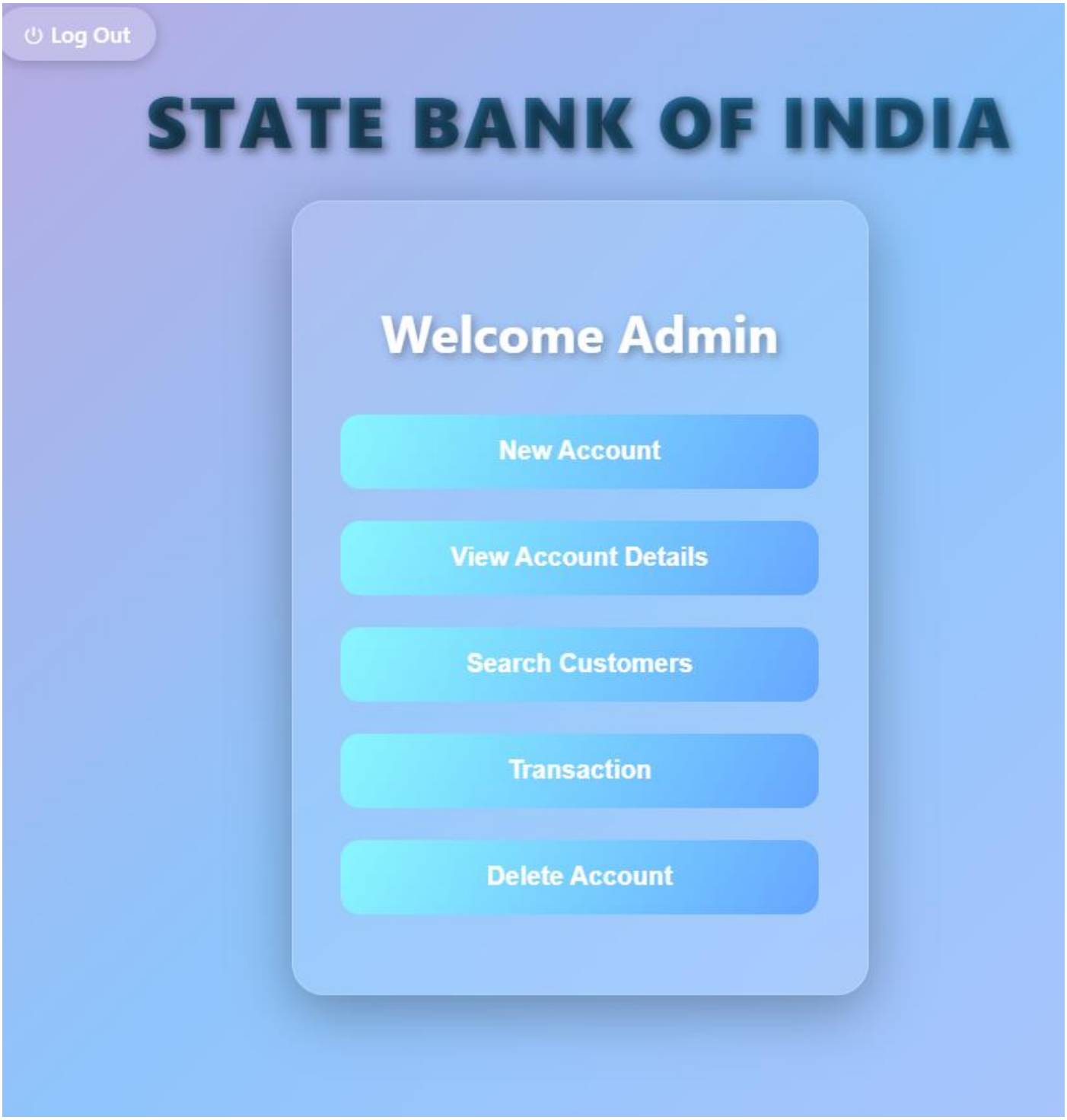
```
  output += "</ul><a href='/dashboard/" + accno + "'>Back</a>";
res.send(output);
});
app.listen(port, () => {
  console.log(Server is running on http://localhost:${port});
});
```

OUTPUT :

Admin login



Admin Interface



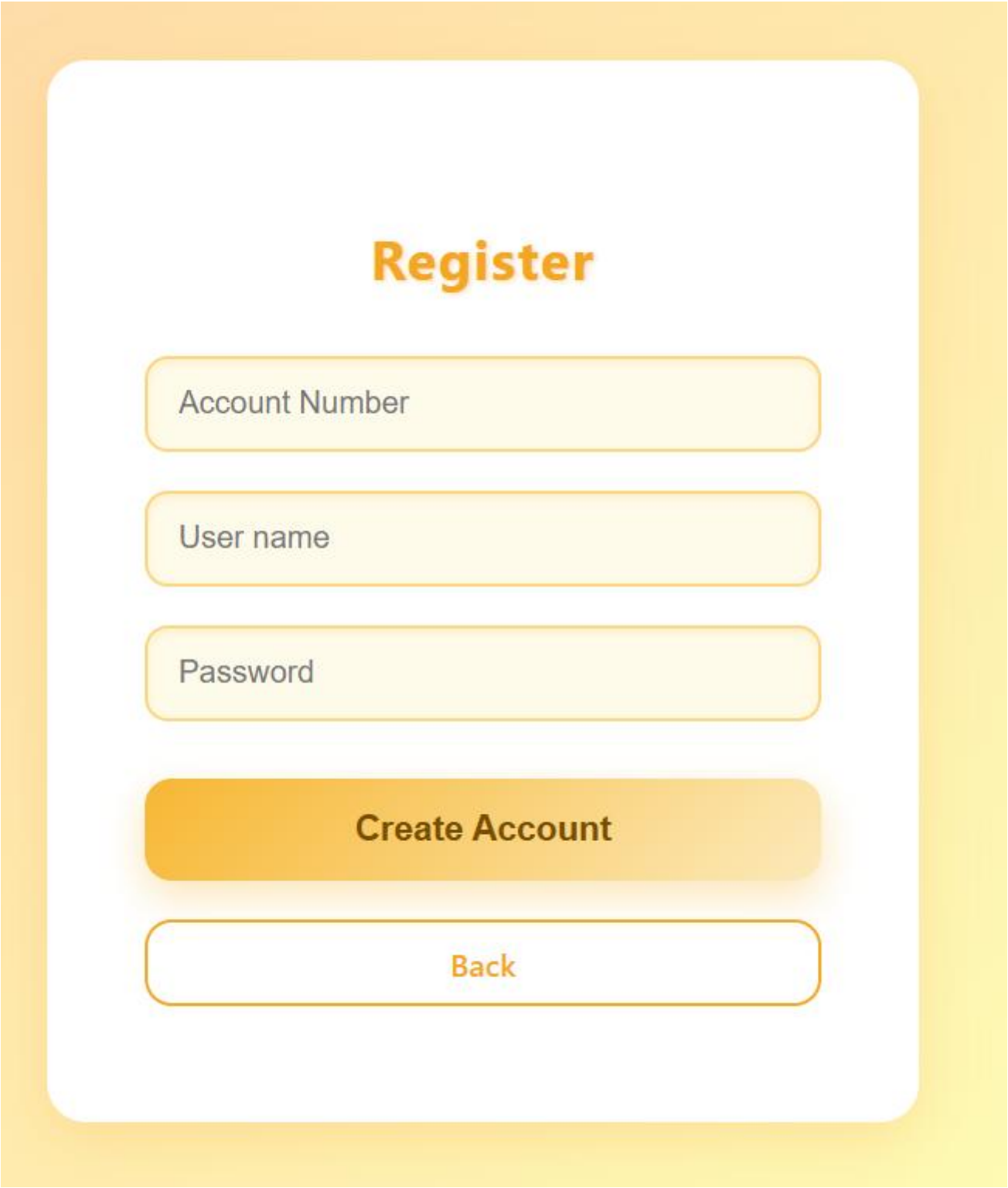
View Account Details

back

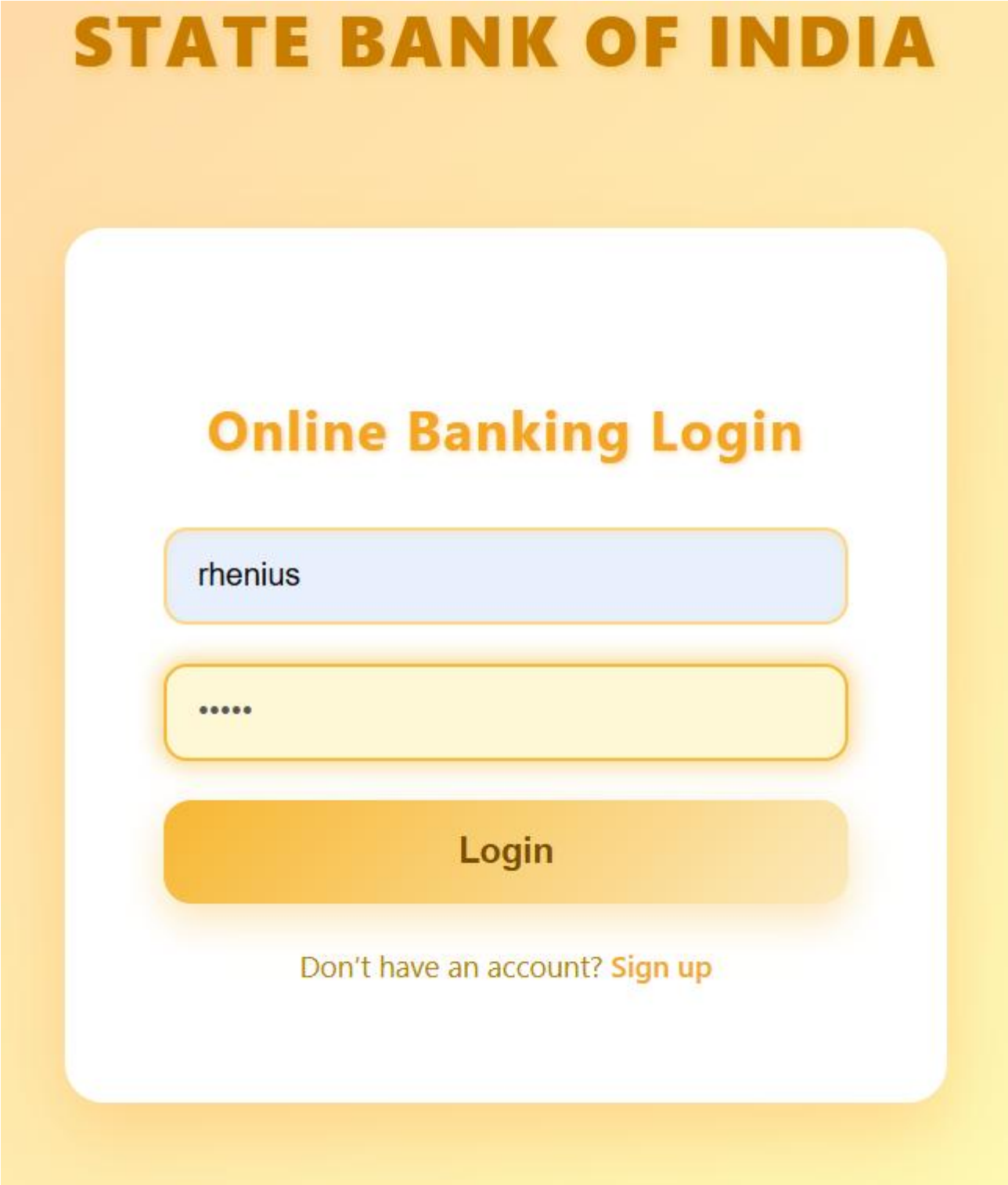
Registered Accounts

Account No	Account Holder Name	Initial Deposit	Account Type
1003	rhenius	49000	Salary
1004	aravind	20000	Fixed
1005	praveen kumar	20000	Current
1002	madhan kumar	21000	Current
1016	rajesh	41000	Current

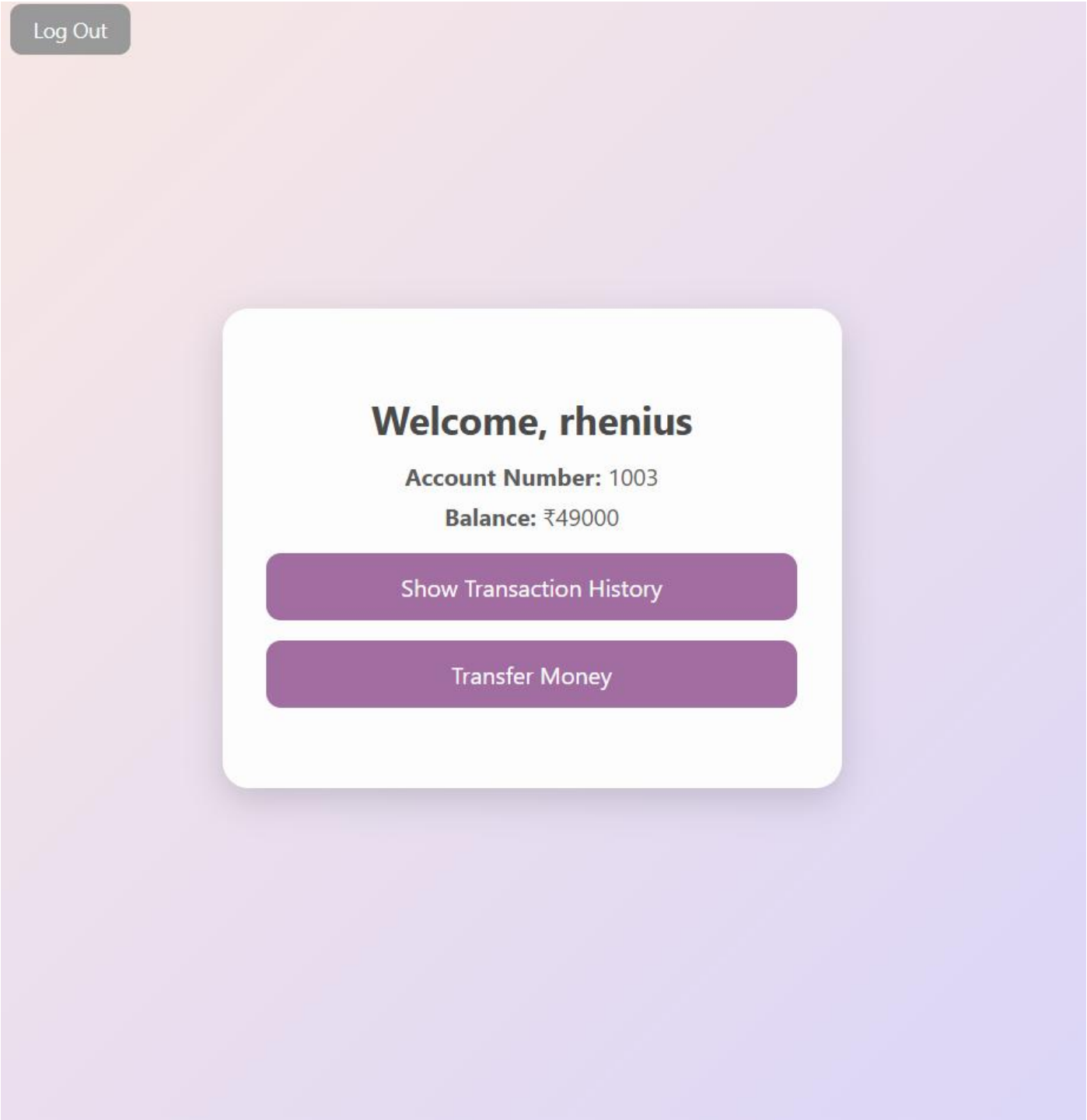
New Registration



User Login


STATE BANK OF INDIA
Online Banking Login
rhenius
•••••
Login
Don't have an account? Sign up

User Interface



User Transaction History

← Back

Transaction History

Date	Type	Amount	Description
7/6/2025, 10:45:00 am	debit	₹1000	Money transferred to account 1002
7/6/2025, 10:44:22 am	debit	₹1000	Cash withdrawal
6/6/2025, 11:08:14 pm	debit	₹100	Money transferred to account 1016
6/6/2025, 11:05:33 pm	credit	₹1000	Money received from account 1016
6/6/2025, 11:04:53 pm	credit	₹100	Money received from account 1016
6/6/2025, 10:55:15 pm	credit	₹2000	Cash deposit
6/6/2025, 10:54:09 pm	credit	₹200	Money received from account 1016
6/6/2025, 10:52:51 pm	debit	₹200	Money transferred to account 1016
6/6/2025, 10:52:35 pm	debit	₹2000	Money transferred to account 1016

Result :

This project successfully demonstrates a working banking system with core functionalities like account creation, login, money transactions, and transaction history — all backed by a MongoDB database and built with a clean UI using HTML and CSS. It simulates real-world banking operations in a simplified form.